BLACK BOX TESTING

2. (7 punti) Definire i test Black Box per la seguente funzione, usando equivalence classes e boundary conditions. La funzione riceve un numero e restituisce `true` se il numero è primo, `false` altrimenti. Attenzione che in typescript `number` può contenere anche numeri con la virgola.

```
function isPrime(numero: number): Boolean
```

Criteri

- Segno del numero
- Numero primo o non primo

Classi di equivalenza:

- Segno del numero > 0
- Segno del numero < 0
- È primo
- Non è primo

segno	primo	Test Case	
negativo	-	T1(-10, false) TB1(-lowestNumber, false) TB2(-1, false) TB3(0, false)	
positivo	si	T2(2, true) TB3(1, false) TB4(+highestNumber, ?) TB5(+highestNumber +1, ?) TB6(+highestNumber -1, ?)	In TB4, TB5, TB6 il risultato vero o falso deve essere calcolato conoscendo il valore effettivo di highestNumber TB3 può essere considerato limite sia nel senso del segno (0-1,0, 0+1) sia nel senso di primo (1 è un particolare numero non primo, poiché ha un solo fattore nella divisione, mentre la definizione di primo richiede un minimo di 2 max 2 fattori nella divisione).
positivo	no	T3(4, false)	

2 Define black box tests for the following function, using equivalence classes and boundary conditions.
The function receives a number and returns true if the number is prime

function isPrime(number: number): boolean

```
criteria: sign of number, prime or not
classes: sign of number > 0, sign < 0
         is prime, ! is prime
```

sign	Is prime	Test cases	
negative	-	T1(-10, error) Tb1(-lowestinteger, error) Tb2(-1, error) Tb2(0, error)	
positive	Yes	T2(2, true) Tb3(1, false) Tb4(+highestInteger, ?) Tb5(+highestInteger +1, ?) Tb6(+highestInteger -1, ?)	T2, Tb3, Tb4, Tb6 result true or false should be computed knowing the actual value of highestInteger Tb3 can be considered boundary both in sense of sign (0-1,0, 0+1) and in sense of prime (1 is a particular non prime number, since it has only one factor in division, definition of prime requires min 2 max 2 factors in division)
positive	Not	T3(4, false)	

2 Define black box tests for the following function, using equivalence classes and boundary conditions. The function receives a number and returns true if the number is prime

function `isPrime(number: number): boolean`

```
criteria: sign of number, prime or not
classes: sign of number > 0, sign < 0
          is prime, ! is prime
```

sign	Is prime	Test cases	
negative	-	T1(-10, error) Tb1(-lowestinteger, error) Tb2(-1, error) Tb2(0, error)	
positive	Yes	T2(2, true) Tb3(1, false) Tb4(+highestInteger, ?) Tb5(+highestInteger +1, ?) Tb6(+highestInteger -1, ?)	Tb4, Tb5, Tb6 result true or false should be computed knowing the actual value of highestInteger Tb3 can be considered boundary both in sense of sign (0-1,0, 0+1) and in sense of prime (1 is a particular non prime number, since it has only one factor in division, definition of prime requires min 2 max 2 factors in division)
positive	Not	T3(4, false)	

Define black box tests for the following function, using equivalence classes and boundary conditions.

double computeBillAmount(int contractType, int consDay, int ConsNight)

The function computes the amount of an electric bill during a month period.
contractType can be 1 or 2,
consDay is the number of KWatt hours (KWh) consumed during the day in the month period
consNight is the number of KWh consumed during the night in the month period

The amount is computed as follows:
For contractType 1 the cost of Kwh day is 1, the cost of Kwh night is 0,5, if the total consumption (consDay + consNight) exceeds 50 then a 10% penalty fee is added
For contractType 2 the cost of Kwh day is 2, the cost of Kwh night is 1, if the total consumption (consDay + consNight) exceeds 100 then a 5% penalty fee is added

Ex computeBillAmount(1, 10, 10) → 15 = (10*1 + 10*0,5)
computeBillAmount(1, 30, 30) → 49.5 = (30*1 + 30*0,5)*1.1 = 45 * 1.1 = 49.5 (penalty applied)
computeBillAmount(2, 10, 10) → 30 = (10*2 + 10*1)
computeBillAmount(2, 60, 60) → 189 = (60*2 + 60*1)*1.05 = 180 * 1.05 = 189 (penalty applied)

Contract Type	consDay	consNight	consDay+consNight	Test case
[minint, 0]	-	-	-	(-50, 10, 10) → error
1	[minint, 0[	-	-	(1, -30, 30) → error
	[0, maxint]	[minint, 0[	-	(1, 30, -30) → error
		[0, maxint]	[0, 50]	(1, 10, 10)
			[51, 100]	(1, 30, 30)
			[101, maxint]	(1, 30, 80)
2	[minint, 0[	-	-	(2, -30, 30) → error
	[0, maxint]	[minint, 0[	-	(2, 30, -30) → error
		[0, maxint]	[0, 50]	(2, 10, 10)
			[51, 100]	(2, 30, 30)
			[101, maxint]	(2, 30, 80)
[3, maxint]	-	-	-	(4, 10, 10) → error

Boundary condition tests
ContractType: minint-1, minint, minint-1, -1, 0, 1,2, 3, maxint-1, maxint, maxint+1
consDay : minint-1, minint, minint-1, -1, 0, 1, maxint-1, maxint, maxint+1
consNight : same as consDay
consDay+consNight : minint-1, minint, minint-1, -1, 0, 1, 49, 50, 51, 99, 100, 101, 102, maxint-1, maxint, maxint+1

Define black box tests for the following function, using equivalence classes and boundary conditions.

int computeTax (int totalIncome, int deductible, int taxDeductible)

The function computes the amount of money to be paid as taxes by a person, in euro. All values are integers, cents are rounded down.
totalIncome is the total amount of money gained by a person in a fiscal year. The tax is computed as 30% of totalIncome – deductible, however the first 10.000 euro are tax free.

Ex computeTax (10000, 0,0) → 0
computeTax(20000, 0,0) → 3.000 (30% of 20.000- 10.000)

deductible is the amount of expenses that the person declares. The deductible is subtracted by totalIncome, however the maximum deductible per year is 4000. In any case the tax to be paid cannot be negative

computeTax(20000, 2000, 0) → 2.400 (30% of (20.000-2000) - 10.000)
computeTax(20000, 5000, 0) → 1.800 (30% of (20.000-4000) - 10.000)

taxDeductible is another category of expenses that the person declares. These expenses are treated in a different way. 20% of these expenses is subtracted to the amount of taxes due. However this applies only to a max of 2000 taxDeductible per year

computeTax(20000, 0, 1000) → 2.800 = 3000 (30% of (20.000-0) - 10.000) - 200 (20% of 1.000)
computeTax(20000, 0, 3000) → 2.600 = 3000 (30% of (20.000-0) - 10.000) - 400 (20% of 2000)

total income [minInt, 0[, [0, 10.000], [10.000, maxInt]
deductible [minInt, 0[, [0, 4.000],] 4.000, maxInt]
taxDeductible [minInt, 0[, [0, 2.000],]2.000, maxInt]

Total income	deductible	taxDeductible	0,3(totalIncome-10000-deductible)-0,2*taxDeductible>=0	V / I	Test cases
[minInt, 0[	[minInt, 0[	[minInt, 0[	-	I	(-1, -1. -1) error
[0, 10.000]	[0, 4.000]	[0, 2.000]	T	V	(1, 4000, 0), 0 (1, 0, 2000), 0 (1, 4000, 2000), 0
			F	V	(1000, 2000, 1000),
		]2.000, maxInt]	T	V	(1,0, 4000) , 0 (1, 4000, 4000), 0
			F	V	(1000, 2000, 4000),
	]4.000, maxInt]	[0, 2.000]	T	V	
			F	V	
		]2.000, maxInt]	T	V	
			F	V	
]10.000, maxInt]	[0, 4.000]	[0, 2.000]	T	V	

Define black box tests for the following function, using equivalence classes and boundary conditions.

Int BMI(int sex, int height, int weight)

The function receives the sex of a person (1 man, 2 female), the height in centimeters (170 = 1meter 70 centimers), the weight in kilograms (80 = 80 kilograms). It returns

0 if the input is not acceptable
1 if the BMI is normal
2 if the BMI is below threshold
3 if the BMI is above threshold

The BMI is = weight / (height * height) and the normal ranges are 18-25 for men, 17-24 for women.

sex	height	weight	BMI	Valid / invalid	Test case
[minint, 0]	-	-	-	I	T(-10, 20,20) → 0
[3 maxint]	-	-	-	I	T(11, 20,20) → 0
-	[minint, 0]	-	-	I	T(1, -10, 20) → 0
-	[300, maxint]	-	-	I	T(1, 400, 20) → 0
-	-	[minint 0]	-	I	T(1, 100, -20) → 0
-	-	[500, maxint]	-	I	T(1, 100, 2000) → 0
1	[1, 299]	[1,499]	Normal	V	T(1, 180, 75) → 1
			Below	V	T(1, 180, 50) → 2
			above	V	T(1, 180, 300) → 3
2	[1, 299]	[1,499]	Normal	V	T(2, 180, 75) → 1
			Below	V	T(2, 180, 50) → 2
			above	V	T(2, 180, 300) → 3

Boundaries to be added

2 black box

Define black box tests for the following function, using equivalence classes and boundary conditions.

boolean canGoTo(int charge, int movingMode, int distance)

The function receives the charge of the battery (can be 0 to 100), the mode (0 slow, 1 fast), the distance to be travelled (from 0 to maxint). The function returns true if the distance can be traveled by the robot, false otherwise. The formula applied is: 1 unit of charge per unit of distance in slow mode, 2 units of charge per unit of distance in fast mode.

Ex canGoTo(100, 0, 50) → true (charge available to go until distance 100/ 1 = 100 >= 50)
canGoTo(100, 1, 60) → false (charge available to go until distance 100/ 2 = 50 < 60)

Charge	Moving mode	distance	Valid / Invalid
[Minint, 0[	[minint, 0[	[minint, 0[	I
		[0, maxint]	I
	0	[minint, 0[	I
		[0, maxint]	I
	1	[minint, 0[	I
		[0, maxint]	I
	[2, maxint]	[minint, 0[	I
		[0, maxint]	I
[0, 100]	[minint, 0[	[minint, 0[	I
		[0, maxint]	I
	0	[minint, 0[	I
		[0, maxint]	V
	1	[minint, 0[	I
		[0, maxint]	V
	[2, maxint]	[minint, 0[	I
		[0, maxint]	I
]100, maxint]	[minint, 0[	[minint, 0[	I
		[0, maxint]	I
	0	[minint, 0[	I
		[0, maxint]	I
	1	[minint, 0[	I
		[0, maxint]	I
	[2, maxint]	[minint, 0[	I
		[0, maxint]	I

2 (8 points) -Define black box tests for the following function

int twoCirclesIntersectionPoints(int radius1, int x1, int y1, int radius2, int x2, int y2);

The function receives the coordinates of two circles on a plane (as radius and position of the center). The function returns an integer considering the relative positions of the circles: 0 if they have no points in common, 1 if they have one point in common (tangent circles), 2 if they have two points in common (secant), 3 if they have all points in common.

Ex. twoCirclesIntersectionPoints(1, 0, 0, 1, 2, 0) → 1
Ex. twoCirclesIntersectionPoints(1, 0, 0, 1, 1, 0) → 2

Criterion	Valid class	Invalid class	Boundary Condition
Radius sign	positive	Negative	Zero maxint
X y sign	Positive, negative		Maxint, minint
Input parameters type	integer	All others (char, float, double ..)	
Points in common	Zero Zero and one circle is inside the other Zero and one circle is outside the other		
	One One and one circle inside the other One and one circle outside the other		
	Two		
	All in common (the 2 circles are the same circle)		

2 (8 points) -Define black box tests for the following function

double averagePositive (int array[]);

The function receives an array of integers, and computes the average, but considering only the positive numbers.

Ex. averagePositive ({1,2,3,4}) → 2.5
Ex. averagePositive ({1,-2,3}) → 2

Criterion	Valid class	Invalid class	Boundary Condition
Number of elements in array	1 and more	zero	1, infinite
Sign of elements	All positives Positives and negatives	All negatives (result is error or zero? To be clarified)	
Type of elements in array	int	Char, string, float, double	
Range or each element in array	Minint to maxint	>maxint <minint	Minint, minint +1, minint -1 Maxint, maxint -1, maxint +1

2a (4 points) -Define black box tests for the following function

boolean authorizeRental(int nDroppedRequests, int nDroppedReservations);

the function defines if a rental (as in ex 1) can be authorized, with these rules
-if nDroppedRequests > 5, no authorize (returns false)
-if nDroppedReservations > 5, no authorize (returns false)
-if nDroppedRequests + nDroppedReservations > 7, no authorize (returns false)
-else authorize

Criterion	**Valid Class**	**Invalid Class**	**Boundary conditions**	**Tests**
Sign of parameters	Positive	Negative	0	Valid T0) aR(1, 1) , true Invalid T1) aR (-1, 0), false Boundaries T2) aR(0, 0) , true T3) aR(1 ,0) , true T1, false
Range of parameters	0 to 5 (boundaries included)	<0 , >5	0, 5	Valid T4) aR(3, 2) , true Invalid T5) aR(-2,4) , false T6) aR(6, 4) , false Boundaries 0 T1, false T2, true T3, true 5 T7) aR(4, 2) , true T8) aR (5,2) ,true T6, false
Sum of parameters	0 to 7 (boundaries included)	<0, >7	0, 7	Valid T4, true Invalid T9) aR(4,5) , false
				Boundaries 0 T1, false T2, true T3, true 7 T7, true T8, true T10) aR(4,4) , false
Type of parameters	int	Other types		Valid T1, true Invalid T11, aR ('c','d'), error

Esercizio 1 – acceptableToEat

La funzione `acceptableToEat` riceve il peso in grammi di carboidrati, proteine e grassi in una porzione di cibo. Restituisce `true` se entrambe le seguenti condizioni sono soddisfatte:

- la quantità totale di calorie è < 1000
- `(carb + protein) / fat > ½`

Definizione della funzione

```
function acceptableToEat(carb: number, protein: number, fat: number): boolean
```

Esempi

```
acceptableToEat(100, 100, 100); // false
// Calorie = 100*4 + 100*4 + 100*9 = 1700 > 1000

acceptableToEat(1, 1, 10); // false
// Rapporto = 2 / 10 = 0.2 < 0.5

acceptableToEat(1, 1, 1); // true
// Rapporto = 2 / 1 = 2 > 0.5
```

Criteri di test

- Segno di ciascun input (positivo, zero, negativo)
- Formula 1: calorie < 1000
- Formula 2: (carb + protein) / fat > 0.5

Predicati e Valori Limite

- Carb: -1, 0, MAXINT, MAXINT+1
- Proteine: -1, 0, MAXINT, MAXINT+1
- Grassi: -1, 0, 1, MAXINT, MAXINT+1

Tabella dei test

#	Segno di carb	Segno di proteine	Segno di grassi	Formula1	Formula2	Test	Risultato
TB1	Negativo	Positivo	Positivo	Vero	Vero	acceptableToEat(-1, 10, 10)	false
TB2	Zero	Positivo	Positivo	Vero	Vero	acceptableToEat(0, 10, 10)	true
TB3	Positivo	Positivo	Positivo	Falso	Vero	acceptableToEat(2147483647, 10, 10)	false
TB4	Positivo	Positivo	Positivo	Falso	Vero	acceptableToEat(2147483648, 10, 10)	false
TB5	Positivo	Negativo	Positivo	Vero	Vero	acceptableToEat(10, -1, 10)	false
TB6	Positivo	Zero	Positivo	Vero	Vero	acceptableToEat(10, 0, 10)	true
TB7	Positivo	Positivo	Positivo	Falso	Vero	acceptableToEat(10, 2147483647, 10)	false
TB8	Positivo	Positivo	Positivo	Falso	Vero	acceptableToEat(10, 2147483648, 10)	false
TB9	Positivo	Positivo	Negativo	Vero	Falso	acceptableToEat(10, 10, -1)	false
TB10	Positivo	Positivo	Zero	Vero	Vero	acceptableToEat(10, 10, 0)	true
TB11	Positivo	Positivo	Positivo	Vero	Vero	acceptableToEat(10, 10, 1)	true
TB12	Positivo	Positivo	Positivo	Falso	Falso	acceptableToEat(10, 10, 2147483647)	false
TB13	Positivo	Positivo	Positivo	Falso	Falso	acceptableToEat(10, 10, 2147483648)	false
T1	Positivo	Positivo	Positivo	Falso	Vero	acceptableToEat(100, 100, 100)	false
T2	Positivo	Positivo	Positivo	Vero	Falso	acceptableToEat(1, 1, 10)	false
T3	Positivo	Positivo	Positivo	Vero	Vero	acceptableToEat(1, 1, 1)	true

Esercizio 2 – computeFee (noleggio bicicletta)

La funzione `computeFee` calcola (in **centesimi di euro**) il costo per il noleggio di una bicicletta, utilizzando i seguenti parametri:

- `duration` : minuti totali di utilizzo
- `minRate` : costo al minuto (in centesimi) per i minuti **dal 31° al 90° incluso**
- `minRate2` : costo al minuto (in centesimi) per i minuti **oltre il 90°**

Definizione

```
function computeFee(duration: number, minRate: number, minRate2: number): number
```

Regole di calcolo

- Gratis per i primi 30 minuti
- Dal 31° al 90°: minRate cent/min
- Dal 91° in poi: minRate2 cent/min
- Se duration < 0 o minRate o minRate2 negativi → ritorna -1

Criteri

- Segno della durata
- Segno di minRate
- Segno di minRate2
- Intervallo di durata (0–30, 31–90, 91+)

Predicati

- Durata:
  - Positiva (> 0)
  - Zero
  - Negativa
- minRate:
  - Positivo
  - Zero
  - Negativo
- minRate2:
  - Positivo
  - Zero
  - Negativo
- Range durata:
  - 0–30 minuti
  - 31–90 minuti
  - 91+ minuti

Valori limite

```
duration == -1, duration == 0,
duration == 1, duration == 30,
duration == 31, duration == 90, duration == 91

minRate == -1, minRate == 0, minRate == 1

minRate2 == -1, minRate2 == 0, minRate2 == 1
```

#	Segno Durata	Segno minRate	Segno minRate2	Range Durata	Valido	Test	Atteso
TB1	Negativo	Positivo	Positivo	-	NO	computeFee(-1, 10, 20)	-1
TB2	Zero	Positivo	Positivo	0	SÌ	computeFee(0, 10, 20)	0
TB3	Positivo	Positivo	Positivo	0–30	SÌ	computeFee(1, 10, 20)	0
TB4	Positivo	Positivo	Positivo	0–30	SÌ	computeFee(30, 10, 20)	0
TB5	Positivo	Positivo	Positivo	31–90	SÌ	computeFee(31, 10, 20)	10
T1	Positivo	Positivo	Positivo	31–90	SÌ	computeFee(35, 10, 20)	50
T2	Positivo	Positivo	Positivo	31–90	SÌ	computeFee(60, 10, 20)	300
T3	Positivo	Positivo	Positivo	31–90	SÌ	computeFee(65, 10, 20)	350
TB6	Positivo	Positivo	Positivo	31–90	SÌ	computeFee(90, 10, 20)	600
TB7	Positivo	Positivo	Positivo	91+	SÌ	computeFee(91, 10, 20)	620
T4	Positivo	Positivo	Positivo	91+	SÌ	computeFee(95, 10, 20)	700
T5	Positivo	Zero	Positivo	31–90	SÌ	computeFee(65, 0, 20)	0
T6	Positivo	Positivo	Zero	91+	SÌ	computeFee(95, 10, 0)	600
T7	Positivo	Zero	Zero	91+	SÌ	computeFee(95, 0, 0)	0

T8	Positivo	Zero	Positivo	91+	SÌ	computeFee(95, 0, 10)	50
T9	Positivo	Positivo	Negativo	91+	NO	computeFee(95, 10, -10)	-1
T10	Positivo	Negativo	Positivo	91+	NO	computeFee(95, -10, 10)	-1
T11	Positivo	Negativo	Negativo	91+	NO	computeFee(95, -10, -10)	-1
T12	Positivo	Positivo	Positivo	91+	SÌ	computeFee(95, 1, 1)	65
TB8	Positivo	Negativo	Negativo	0–30	NO	computeFee(30, -1, -1)	-1
TB9	Zero	Negativo	Negativo	0	NO	computeFee(0, -1, -1)	-1
T13	Positivo	Positivo	Zero	91+	SÌ	computeFee(91, 1, 0)	60

Esercizio 3 – computeFee (offerta ferroviaria per minori)

Una compagnia ferroviaria offre la possibilità ai minori di 15 anni di viaggiare gratis. L'offerta è dedicata a gruppi da **2 a 5 persone** che viaggiano insieme.
Per essere idonei all'offerta, **almeno un membro del gruppo deve avere almeno 18 anni**.
Se questa condizione è soddisfatta, **tutti i membri sotto i 15 anni viaggiano gratis**, e gli altri pagano il prezzo base.

Definizione

```
function computeFee(basePrice: number, n_passengers: number, n_over18: number, n_under15:
```

Esempi

```
computeFee(20.0, 3, 0, 1); // 60.0 → non eleggibile
computeFee(30.0, 5, 1, 2); // 90.0 → 3 pagano
```

Criteri

- Numero totale di passeggeri
- Numero di passeggeri ≥ 18 anni
- Numero di passeggeri < 15 anni

- Prezzo base

Predicati

- n_passengers:
  - $< 2 \rightarrow$ gruppo non valido
  - 2–5 → valido
  - $> 5 \rightarrow$ errore
- n_over18:
  - $0 \rightarrow$ non ammissibile per l'offerta
  - $\geq 1 \rightarrow$ ammissibile
  - $< 0 \rightarrow$ errore
- n_under15:
  - $0 \rightarrow$ tutti pagano
  - $\geq 1 \rightarrow$ i <15 non pagano
  - $< 0 \rightarrow$ errore
- basePrice:
  - $< 0 \rightarrow$ errore
  - $= 0 \rightarrow$ gratis
  - $> 0 \rightarrow$ normale

Limiti

- Passeggeri: -1, 0, 1, 2, 5, 6
- Maggiorenni: 0, 1
- Minorenni: 0, 1
- Prezzo: 0.0, 1.0

* gli errori sono rappresentati nella forma `-1`

#	Base price	N_passeggeri	18yo+	15yo-	Eleggibile	Test	Risultato Atteso
TB1	20.0	-1	1	1	NO	computeFee(20.0, -1, 1, 1)	Errore
TB2	20.0	0	1	1	NO	computeFee(20.0, 0, 1, 1)	Errore
TB3	20.0	1	1	0	NO	computeFee(20.0, 1, 1, 0)	Errore
TB4	20.0	2	0	1	NO	computeFee(20.0, 2, 0, 1)	40.0
T1	20.0	2	1	1	SÌ	computeFee(20.0, 2, 1, 1)	20.0
T2	20.0	2	1	0	SÌ	computeFee(20.0, 2, 1, 0)	40.0
TB5	20.0	5	1	4	SÌ	computeFee(20.0, 5, 1, 4)	20.0
TB6	20.0	6	1	2	NO	computeFee(20.0, 6, 1, 2)	Errore
TB7	0.0	3	1	2	SÌ	computeFee(0.0, 3, 1, 2)	0.0
TB8	1.0	3	1	2	SÌ	computeFee(1.0, 3, 1, 2)	1.0
T3	30.0	5	1	2	SÌ	computeFee(30.0, 5, 1, 2)	90.0
T4	30.0	3	0	1	NO	computeFee(30.0, 3, 0, 1)	90.0